

<style> /* Eine Referenzkarte ist absichtlich dichter gesetzt als Fliesstext: nur so bleiben beide Seiten vollstaendig. Der Block gilt nur fuer dieses Dokument - auf eigene Builds hat er keine Wirkung. Farben, Schrift und Rahmen kommen weiterhin aus dem Theme, hier wird ausschliesslich die Dichte geregelt.

Beide Uebersetzungen tragen denselben Block, bemessen an der laengeren: deutscher Satz braucht fuer dieselbe Aussage rund ein Sechstel mehr Platz. Wer hier lockerer setzt, verliert die zweite Seite - in einer Sprache.

8pt in den Tabellen ist die Untergrenze. Waechst das Schema weiter, ist nicht die Schrift das Stellrad, sondern der Inhalt: die vollstaendige Referenz steht in Anhang A des Handbuchs, hier nur, was man staendig nachschlaegt. */ .chapter-body table { margin: 0.4em 0; font-size: 8pt; } .chapter-body th, .chapter-body td { padding: 0.7mm 2.0mm; } .chapter-body h1 { margin-top: 0; margin-bottom: 0.3em; } .chapter-body h2 { margin-top: 0.75em; margin-bottom: 0.15em; } .chapter-body p { margin: 0.35em 0; } .chapter-body pre { margin: 0.4em 0; } </style>

markpublish.yaml

Oberste Ebene: document, theme, parts, optional templates_dir. Pfade relativ zu dieser Datei. Bauen: markpublish build [datei] -t pdf|html|all.

document

Metadaten und globale Layout-Schalter. Pflicht ist allein title.

Schlüssel	Standard	Bedeutung
title / subtitle	title nötig	Deckblatt, Kopfzeile, Dateiname
author	–	Deckblatt
date / version	auto / –	auto oder today für das Baudatum
language	Systemsprache	Wählt die Beschriftungen, sonst en
cover / header / footer	true	Deckblatt, Kopf- und Fußzeile
document_toc	full	Großes Verzeichnis: none, full oder Tiefe
part_toc / chapter_toc	full	Trennseiten-Verzeichnisse, gleiche Formen
autonum_style	decimal	decimal, roman, legal, none
autonum_from_level	1	Ab welcher Überschriftenebene gezählt wird
autonum_reset / pagenum_reset	false	Zähler bzw. Seitenzahl neu beginnen

Außerdem `summary` (Deckblatt), `status`, `copyright` (Fußzeile). Unbekannte Schlüssel gehen als `{{ document.mein_schluesel }}` ans Theme.

parts und chapters

Genau zwei Stufen: `parts`: gliedert, `chapters`: trägt den Inhalt. Kapitel schachteln **nicht** weiter — Tiefe entsteht aus den Überschriften der Datei.

Schlüssel	Gilt für	Bedeutung
<code>title / part</code>	Part	Name des Parts, Pflicht
<code>chapters</code>	Part	Flache Liste der Kapitel, mindestens eines
<code>file</code>	Kapitel	Pfad zur Markdown-Datei
<code>title / subtitle / summary</code>	beide	Überschreiben, was in der Datei steht
<code>break_before</code>	beide	<code>page</code> , <code>divider</code> (Trennseite) oder <code>none</code>
<code>document_toc</code>	beide	Anteil am Verzeichnis vorn
<code>part_toc / chapter_toc</code>	Part / Kapitel	Verzeichnis auf der Trennseite
<code>autonum_*</code> , <code>pagenum_reset</code>	beide	Wie unter <code>document</code> , hier lokal

`document_toc: 1` listet ein Kapitel ohne seine Unterüberschriften; am Part gesetzt, flacht es alle Anhänge auf einmal ab. `autonum_*` und `document_toc` vererben von `document` über den Part zum Kapitel — der tiefere Wert gewinnt.

Templates und Themes

Alles Sichtbare steckt in einem Theme: Seitenlayout, Stylesheet und statische Texte. Mitgeliefert wird eines, `default`, in den Varianten `pdf` und `html`.

Wo ein Theme gesucht wird

Der erste Treffer gewinnt — ein Theme kommt immer aus genau einer Quelle, nie gemischt.

Rang	Ort
1. Benutzer	<code>~/.markpublish/templates</code> , sonst das Konfigverzeichnis des Systems
2. Projekt	<code>templates_dir</code> in der YAML, <code>MARKPUBLISH_TEMPLATES_DIR</code> oder <code>./templates</code>
3. Paket	die eingebauten Themes

`markpublish templates` zeigt, was vorhanden ist und woher es stammt.

Anpassen

```
markpublish export-template default ./templates
```

Kopiert das eingebaute Theme ins Projekt, wo Stufe 2 es beim nächsten Build aufgreift. Ein Theme enthält `layout.html`, `styles.css`, `cover.html`, `toc.html`, `chapter_divider.html`, `part_divider.html` und `i18n.yaml`. Markup und Stylesheet laufen beide durch Jinja2 und sehen `document`, `content_items`, `toc_tree` und `labels`.

Statische Texte

Labels lösen über drei Ebenen auf; eine tiefere überschreibt nur die Schlüssel, die sie tatsächlich setzt.

Ebene	Datei
1. Programm	<code>markpublish/i18n.yaml</code>
2. Theme	<code><templates>/<theme>/i18n.yaml</code>
3. Zielformat	<code><templates>/<theme>/<pdf html>/i18n.yaml</code>

Jede Datei ist nach Sprachcode gegliedert; der Schlüssel "*" gilt für jede Sprache. Ein Dokument kann Labels nicht überschreiben — dafür ist das Theme da. Was ein Theme geändert hat, zeigt `markpublish labels --overridden`.